Ratiba Benzerouali

Document technique -

Requêtez une base de données avec SQL

1.Introduction

L'objectif de ce projet était de construire une base de données relationnelle à partir de deux fichiers CSV (region.csv et contrat.csv) afin d'analyser les contrats d'assurance selon différents critères (région, type de logement, formule, etc.).

Le travail s'est déroulé en plusieurs étapes :

- Exploration et nettoyage des données sous Excel
- Création du dictionnaire et du schéma relationnel
- Importation dans MySQL Workbench
- Écriture et exécution de requêtes SQL pour obtenir des analyses pertinentes.

2. Méthodologie

Analyse des fichiers sources sous Excel -> Observation du contenu, des valeurs et des incohérences.

Nettoyage et préparation des données -> Gestion des cellules vides et cohérence des formats.

Création du dictionnaire de données -> Identification des types et contraintes de chaque champ.

Création du schéma relationnel sur Dbdiagram -> Définition des relations entre les tables region et contrat.

Importation dans MySQL Workbench -> Création des tables, import des CSV et vérification du chargement.

Écriture et test des requêtes SQL -> Analyses de base : comptages, regroupements et jointures.

3. Analyse exploratoire (Excel)

L'analyse exploratoire a permis de comprendre la structure des fichiers avant l'importation.

- **Ouverture des fichiers** : Contrat.csv et Region.csv.
- **Observation des colonnes** : typologie, cohérence, formats.
- **Activation des filtres** :

- Détection de cellules vides (notamment dans No_voie, B_T_Q, Type_de_voie).
- Vérification des doublons (aucun doublon sur Contrat_ID).
- Repérage d'éventuelles fautes de frappe ou incohérences (ex. MAISONN, appartemennt).

Ces vérifications ont guidé le choix des **types SQL** et des **contraintes** dans la base.

4. Dictionnaire de données

Le dictionnaire de données sert à **expliquer clairement ce que contient chaque table et chaque colonne**.
Il indique le **type d'information**, le **format** (nombre, texte, etc.) et les **règles** à respecter.
Cela permet de **mieux comprendre la base de données** et d'éviter les erreurs quand on fait des requêtes.

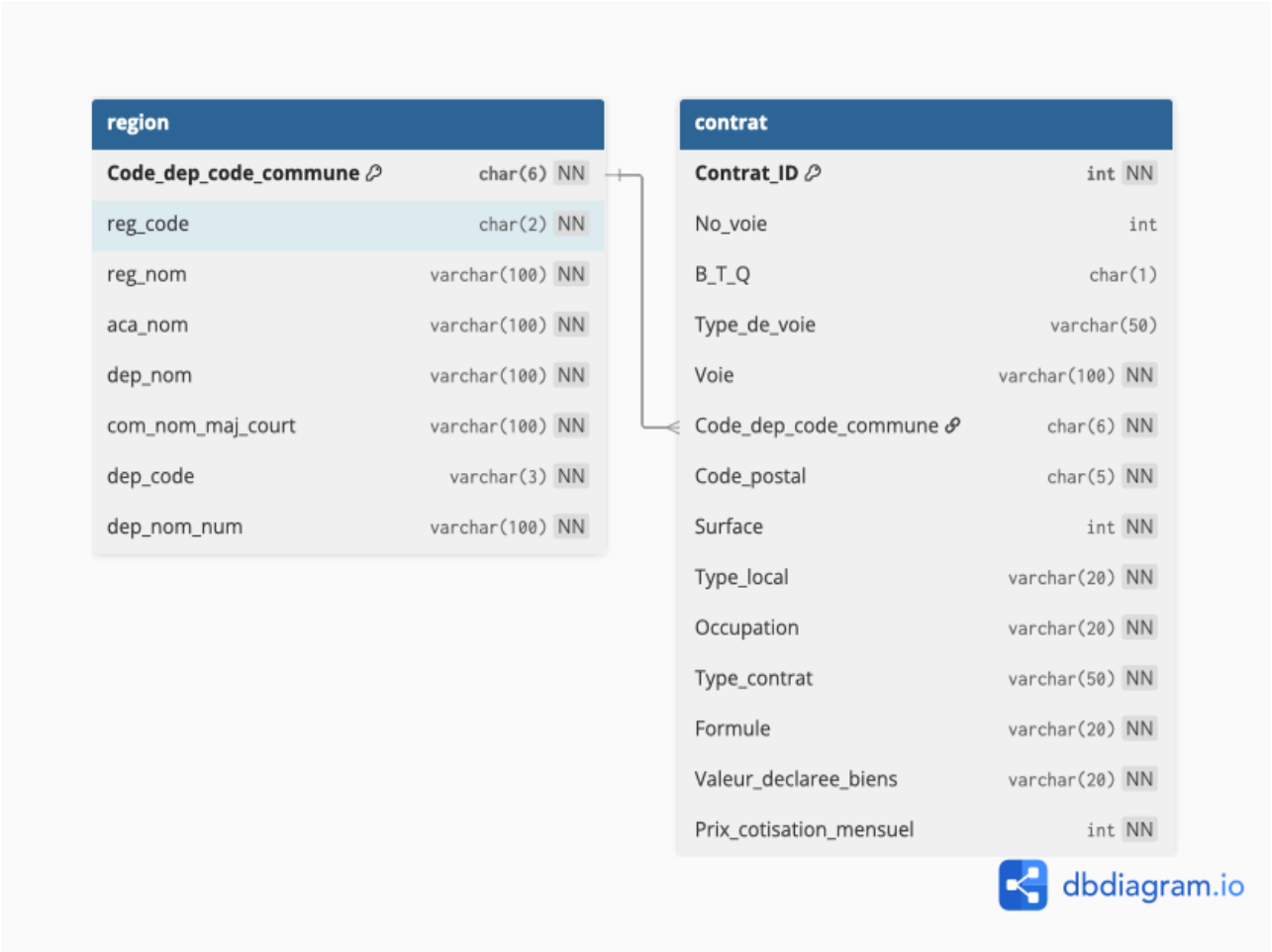
Table region

Colonne	Type	Contraintes	Description
Code_dep_code_commune	CHAR(6)	PRIMARY KEY	Identifiant unique (code département + commune)
reg_code	CHAR(2)	NOT NULL	Code de la région
reg_nom	VARCHAR(100)	NOT NULL	Nom complet de la région
aca_nom	VARCHAR(100)	NULL	Nom de l'académie
dep_nom	VARCHAR(100)	NOT NULL	Nom du département
com_nom_maj_court	VARCHAR(100)	NOT NULL	Nom de la commune en majuscules
dep_code	VARCHAR(3)	NOT NULL	Code du département
dep_nom_num	VARCHAR(100)	NULL	Nom du département avec code numérique

Table contrat

Colonne	Type	Contraintes	Description
Contrat_ID	INT UNSIGNED	PRIMARY KEY	Identifiant unique du contrat
No_voie	INT	NULL	Numéro dans la rue (facultatif)
B_T_Q	CHAR(1)	NULL	Complément d'adresse (bâtiment, tour, etc.)
Type_de_voie	VARCHAR(50)	NULL	Type de voie (RUE, AV, BD, etc.)
Voie	VARCHAR(100)	NOT NULL	Nom de la rue
Code_dep_code_commune	CHAR(5)	NOT NULL	Code de la commune (lien logique avec region)
Code_postal	CHAR(5)	NOT NULL	Code postal du logement
Surface	INT UNSIGNED	NOT NULL	Surface habitable
Type_local	ENUM('Maison','Appartement')	NOT NULL	Type de logement
Occupation	ENUM('Locataire','Proprietaire')	NOT NULL	Statut d'occupation
Type_contrat	ENUM('Residence principale','Residence secondaire','Mise en location')	NOT NULL	Type de contrat
Formule	ENUM('Classique','Integral')	NOT NULL	Type de formule d'assurance
Valeur_declaree_biens	ENUM('0-25000','25000-50000','50000-100000','100000+')	NOT NULL	Valeur déclarée du bien
Prix_cotisation_mensuel	INT UNSIGNED	NOT NULL	Montant mensuel de cotisation

5. Schéma relationnel



Relation **1 → N** entre region et contrat :

- Une région (ou commune) peut regrouper plusieurs contrats.
- Un contrat est rattaché à une seule région (ou commune).
- Clé étrangère : contrat.Code_dep_code_commune → region.Code_dep_code_commune.

6. Mise en place de la base de données

- **CREATE DATABASE IF NOT EXISTS** : crée la base assurance si elle n'existe pas déjà (évite les erreurs si on relance le script).
- **CHARACTER SET utf8mb4** : autorise tous les caractères (accents, emoji…).
- **COLLATE utf8mb4_unicode_ci** : définit l'ordre de tri (insensible à la casse, adapté au français).
- **USE assurance** : dit à MySQL de travailler dans cette base pour la suite.

Nettoyage **DROP TABLE IF**

- Supprimer les tables si elles existent déjà.
- Permet de **rejouer le script** proprement sans erreur.

Création de la table Region :

```sql
CREATE DATABASE IF NOT EXISTS assurance CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
-- CHARACTER SET utf8mb4 → permet d'écrire tous les caractères (accents, emojis, etc.).
-- COLLATE utf8mb4_unicode_ci → définit la façon dont MySQL trie les lettres (par ordre alphabétique, sans tenir compte d

USE assurance;

-- 2) Nettoyage
DROP TABLE IF EXISTS contrat;
DROP TABLE IF EXISTS region;

-- 3) Table REGION
CREATE TABLE region (
  Code_dep_code_commune CHAR(6) PRIMARY KEY,
  reg_code CHAR(2) NOT NULL,
  reg_nom VARCHAR(100) NOT NULL,
  aca_nom VARCHAR(100),
  dep_nom VARCHAR(100) NOT NULL,
  com_nom_maj_court VARCHAR(100) NOT NULL,
  dep_code VARCHAR(3) NOT NULL,
  dep_nom_num VARCHAR(100)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
-- moteur MySQL moderne qui gère les relations (clés étrangères) + encodage universel pour les textes.
```

Colonnes importantes :

- **Code_dep_code_commune (CHAR(5), PRIMARY KEY)** : identifiant unique de la commune (clé primaire).
- **reg_code / dep_code** : codes région et département.
- **reg_nom / dep_nom / com_nom_maj_court** : libellés utiles pour les analyses.

Pourquoi ces choix :

- **CHAR(5)** pour le code de commune (format fixe sur 5 caractères).
- **NOT NULL** là où l'info est indispensable.
- **InnoDB + utf8mb4** : moteur moderne + encodage universel.

Création de la table Contrat

```sql
-- 4) Table CONTRAT
CREATE TABLE contrat (
  Contrat_ID INT UNSIGNED PRIMARY KEY,
  No_voie INT,
  B_T_Q CHAR(1), -- B, T ou Q
  Type_de_voie VARCHAR(50), -- ex : RUE, AV, BD, etc.
  Voie VARCHAR(100) NOT NULL,
  Code_dep_code_commune CHAR(5) NOT NULL, -- correspond au code dans REGION
  Code_postal CHAR(5) NOT NULL,
  Surface INT UNSIGNED NOT NULL,
  Type_local ENUM('Maison', 'Appartement') NOT NULL,
  Occupation ENUM('Locataire', 'Proprietaire') NOT NULL,
  Type_contrat ENUM('Residence principale', 'Residence secondaire', 'Mise en location') NOT NULL,
  Formule ENUM('Classique', 'Integral') NOT NULL,
  Valeur_declaree_biens ENUM('0-25000', '25000-50000', '50000-100000', '100000+') NOT NULL,
  Prix_cotisation_mensuel INT UNSIGNED NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Colonnes clés :

- **Contrat_ID (PRIMARY KEY)** : identifiant unique de chaque contrat (empêche les doublons).
- **Code_dep_code_commune (CHAR(5))** : code de commune, **clé de jointure** avec region.
- **ENUM** (Type_local, Occupation, Type_contrat, Formule, Valeur_declaree_biens) :
  limite les valeurs à une **liste validée** par le dictionnaire.
- **Surface / Prix_cotisation_mensuel** en **UNSIGNED** : valeurs **positives** uniquement.

Import automatique du fichier CSV Région :

```sql
-- Import automatiquement du fichier CSV region
LOAD DATA LOCAL INFILE '/Users/ratibabenzerouali/Documents/module_sql/Region.csv'
INTO TABLE region
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ';' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
(
  Code_dep_code_commune,
  reg_code,
  reg_nom,
  aca_nom,
  dep_nom,
  com_nom_maj_court,
  dep_code,
  dep_nom_num
);
```

Ce que ça fait :

- **LOAD DATA LOCAL INFILE** : import rapide depuis un CSV.
- **FIELDS TERMINATED BY ';'** : colonnes séparées par ;.
- **ENCLOSED BY '"'** : champs texte entourés de guillemets.
- **IGNORE 1 LINES** : ignore la ligne d'entête du CSV.

Même logique pour Contrat, avec gestion des champs vides :

```
-- Import automatique du fichier CSV contrat avec nettoyage des données vides.
LOAD DATA LOCAL INFILE '/Users/ratibabenzerouali/Documents/module_sql/Contrat.csv'
INTO TABLE contrat
CHARACTER SET utf8mb4
FIELDS TERMINATED BY ';' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 LINES
) (
  Contrat_ID,
  @No_voie,
  B_T_Q,
  Type_de_voie,
  Voie,
  Code_dep_code_commune,
  Code_postal,
  Surface,
  Type_local,
  Occupation,
  Type_contrat,
  Formule,
  Valeur_declaree_biens,
  @Prix_cotisation_mensuel
 )
SET
  No_voie = NULLIF(@No_voie,''),
  Prix_cotisation_mensuel = NULLIF(@Prix_cotisation_mensuel,'');
-- Si la valeur est vide dans le fichier, remplace-la par NULL au lieu d'un texte vide
```

Pourquoi NULLIF :

- Transforme les **cellules vides** du CSV en **NULL** (au lieu d'une chaîne vide) pour éviter des erreurs de type.

Résultats finaux :

- region → 38 916 lignes importées
- contrat → 30 335 lignes importées

7. Requêtes SQL (tests et analyses)

Quelques requêtes utilisées pour vérifier et analyser les données :

Q1. Contrats à Caen

```
SELECT contrat_id, surface
FROM contrat
JOIN region
ON contrat.Code_dep_code_commune = region.Code_dep_code_commune
WHERE region.com_nom_maj_court = 'CAEN';
```

Q2. Maisons dans le département 71

```
SELECT contrat_id, Type_contrat, Formule
FROM contrat
JOIN region
ON contrat.code_dep_code_commune = region.Code_dep_code_commune
WHERE LOWER(TRIM(contrat.type_local)) = 'maison'
AND region.dep_code = '71'';
```

Q3. Liste des régions sans doublons

```
SELECT DISTINCT region.reg_nom as noms_regions
FROM region;
```

8. Applications et requêtes avancées

Exemples d'analyses menées :

- **Top 5 des plus grandes surfaces**

  ```
  SELECT contrat.Contrat_ID as id_contrat, contrat.Surface AS
  grandes_surfaces
  FROM contrat
  ORDER BY contrat.Surface DESC
  LIMIT 5;
  ```

- **Prix moyen de cotisation mensuelle**

  ```
  SELECT ROUND(AVG(Prix_cotisation_mensuel),1) AS
  prix_moyen_cotisation_mesuelle
  FROM contrat;
  ```

- **Répartition par valeur de biens**

  ```
  SELECT contrat.valeur_declaree_biens as valeur_biens,
  COUNT(contrat.contrat_ID) as nb_contrats
  ```

```
FROM contrat
GROUP BY contrat.valeur_declaree_biens
ORDER BY contrat.valeur_declaree_biens;
```

Analyses complémentaires

Exemples :

- Surface moyenne des contrats à Paris (département 75)
- Classement des départements au prix moyen le plus élevé
- Communes ayant au moins 150 contrats
- Nombre de contrats par région

Ces requêtes m'ont permis de tester GROUP BY, HAVING, ORDER BY, et les **jointures** entre tables.

10. Apports et apprentissages

- Compréhension du **modèle relationnel** et des **clés étrangères**
- Création d'un **dictionnaire de données cohérent**
- Compréhension du **nettoyage et typage des données**
- Application des commandes SQL principales :
  SELECT, JOIN, WHERE, GROUP BY, HAVING, ORDER BY
- Interprétation des résultats
- Vision globale du cycle de la donnée : **Excel → MySQL → Analyse**

11. Conclusion

Ce projet m'a permis de travailler sur une vraie base de données, depuis la préparation jusqu'à l'analyse.

J'ai appris à **nettoyer**, **structurer** et **analyser** les données de façon efficace.

La démarche suivie est claire, organisée et peut être facilement **reproduite** pour d'autres projets.